浙江大学

《计算机组成与设计》
实验报告

实验名称 Cache Organization and Performance Evaluation

姓　　名 _____

学　　号 _____

院　　系 _____

指导老师 刘鹏

实验日期 2024 年 12 月 19 日

1 实验介绍

在这次实验中，首先需要先构建一个模拟缓存器，并验证其正确性。在确保其正确性后，将使用该模拟器进行相关性能评估研究，旨在研究许多不同的缓存组织和管理策略，强化课堂上关于缓存如何工作的理论知识。

模拟器接受一系列对内存的读写操作作为输入，模拟器是可配置的，可以修改缓存大小、块大小，可以选择是否为统一缓存或分离的指令和数据缓存（I-D），以及写回、写穿、写分配或不写分配的写策略。模拟器跟踪指令和数据的引用次数，数据和指令的缺失次数，从内存中获取的字节数以及写入内存的字节数。当关联度大于 1 时，使用的替换策略是最近最少使用（LRU）。

2 理论框架和代码实现

2.1 初始化代码

为了逻辑更加清晰,初始化部分我配置了两个函数,分别为 init_cache_real 函数和 init_cache 函数，共同完成了缓存模拟器的初始化工作。init_cache_real 负责初始化 cache 结构体，具体初始化内容在注释中已经标明；而 init_cache 来负责选择缓存系统，并初始化缓存统计数据, 即将访问次数、未命中次数、替换次数、需求获取次数和回写次数等设为 0。

```
void init_cache(){
if(cache_split){
init_cache_real(&c1, cache_isize);
init_cache_real(&c2, cache_dsize);
// 如果需要使用分离的cache，则c1用作指令cache，c2用作数据cache
 }
else
{init_cache_real(&c1, cache_usize);}
// 把统计量都初始化为0
//指令初始化为0
cache_stat_inst.accesses =0;
cache_stat_inst.misses= 0;
cache_stat_inst.replacements = 0;
cache_stat_inst.demand_fetches = 0;
cache_stat_inst.copies_back = 0;
//数据初始化为0
cache_stat_data.accesses = 0;
cache_stat_data.misses = 0;
```

```
cache_stat_data.replacements = 0;
cache_stat_data.demand_fetches = 0;
cache_stat_data.copies_back = 0;
}

void init_cache_real(cache *c, int size)
{//该函数根据提供的 size 参数初始化缓存结构体 c 的各个字段
//设置缓存的参数
//并为缓存的每个集合分配内存空间来存储缓存行的信息
//真正初始化cache结构体代码
c->size = size;//设置缓存的总大小
c->associativity = cache_assoc;
//设置缓存的关联度，即每个集合中可以有多少行
c->n_sets = size/(cache_assoc * cache_block_size);
//set数 = cache的大小/每个set的大小
c->LRU_head = (Pcache_line *)malloc(sizeof(Pcache_line)*(c->n_sets));
//为LRU链表分配内存
c->LRU_tail = (Pcache_line*)malloc(sizeof(Pcache_line)*(c->n_sets));
c->set_contents = (int*)malloc(sizeof(int)*(c->n_sets));
//为集合内容计数器分配内存
c->contents = 0;//设置缓存的初始内容计数
c->index_mask_offset = LOG2(cache_block_size);
c->index_mask = ((1<<LOG2(c->n_sets))-1) << c->index_mask_offset ;
for(int i=0; i< c->n_sets; i++)//初始化LRU链表
{
c->LRU_head[i] = NULL;
c->LRU_tail[i] = NULL;
c->set_contents[i] = 0;
}
}
```

2.2 清空部分代码

对于清空部分，和初始化代码一样，配置了 flush 和 flush_cache 两个函数，负责清空缓存，flush 函数选择要清空的 cache，flush_cache 对即将缓存中的所有内容移除，如果启用了写回策略且缓存行标记为脏，则增加回写统计数据。

```
void flush()
{
  flush_cache(&c1);
    if (cache_split) {
        flush_cache(&c2);
}
}
void flush_cache(cache *c) {
    int block_size_in_words = cache_block_size / WORD_SIZE;
    for (int i = 0; i < c->n_sets; i++) {
        Pcache_line cache_line = c->LRU_head[i];
        while (c->set_contents[i] > 0) {
            if (cache_writeback && cache_line->dirty) {
                cache_stat_data.copies_back += block_size_in_words;
            }
            delete(&c->LRU_head[i], &c->LRU_tail[i], cache_line);
            c->set_contents[i]--;
            cache_line = c->LRU_head[i];
        }
    }
    c->contents = 0;
}
```

2.3 重要代码思路及实现

对于这部分我的思路是这样的，首先对于所有的 cache，都应该有以下搜索操作，遍历 LRU 链表，查找标签匹配的缓存行。如果缓存命中，就更新访问统计数据，并调整 LRU 顺序，将命中的缓存行移动到 LRU 链表的头部。如果缓存未命中，则需更新未命中统计数据，创建新的缓存行，然后就要分两种情况：如果当前集合中的缓存行数小于关联度（说明还有空间），就直接将新缓存行插入 LRU 链表；如果当前集合已满，需要回写脏缓存行，替换最不常用的缓存行。

因此这部分我配置了三个函数,perform_access、handle_cache_miss 和 create_new_cache_line。这些函数共同处理缓存访问。

perform_access 函数为该部分主函数，主要是选择目标缓存，从地址中提取索引和标签，遍历目标缓存的 LRU 链表，查找标签匹配的缓存行，处理缓存命中的情况；如果缓存未命中，调用 handle_cache_miss 函数处理。

handle_cache_miss 函数用于处理未命中情况，需要更新未命中统计数据, 处理回写回写统计，创建新的缓存行并根据情况选择如何插入。

create_new_cache_line 函数用于创建一个新的缓存行。分配内存并初始化新缓存行。

```
void perform_access(unsigned addr, int access_type) {
    Pcache target_cache;
    int index;
    unsigned tag;
    int found = 0;
    Pcache_line cache_line, prev_line = NULL;

    // 根据是否分离缓存选择目标缓存
    if (cache_split) {
        target_cache = (access_type == INST_LOAD) ? &c1 : &c2;
    } else {
        target_cache = &c1;
    }

    // 计算索引和标签
    index = (addr & target_cache->index_mask) >> target_cache->index_mask_offset;
    tag = addr >> (LOG2(target_cache->n_sets) + target_cache->index_mask_offset);

    // 遍历缓存行查找匹配的标签
     cache_line = target_cache->LRU_head[index];
    while (cache_line) {
        if (cache_line->tag == tag) {
            // 缓存命中
            if (access_type == DATA_ACCESS || access_type == DATA_STORE) {
                cache_stat_data.accesses++;
            } else if (access_type ==INST_LOAD) {
                cache_stat_inst.accesses++;}

            if (access_type ==  DATA_STORE && !cache_writeback) {
                    //判断 增加回写统计
```

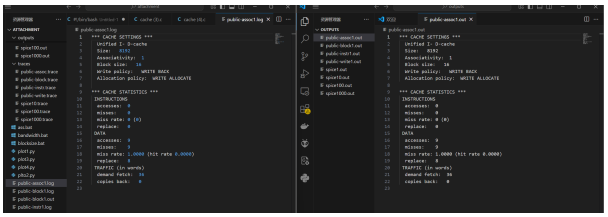
```
                    cache_stat_data.copies_back++;
                }
            //更新 LRU
            delete(&target_cache->LRU_head[index], &target_cache->LRU_tail[index], cache_line);
            insert(&target_cache->LRU_head[index], &target_cache->LRU_tail[index], cache_line);
             if (access_type ==  DATA_STORE ) {
                    cache_line->dirty=1; //设置行脏
                }

            found = 1;
            break;
        } else {
            prev_line = cache_line;
            cache_line = cache_line->LRU_next;
        }
    }

    // 处理缓存未命中
    if (found==0) {
        handle_cache_miss(addr, tag, index, target_cache, access_type);
    }
}


void handle_cache_miss(unsigned addr, unsigned tag, int index, Pcache target_cache, int access_type) {

    if (access_type == DATA_ACCESS || access_type == DATA_STORE) {
        cache_stat_data.misses++;
        cache_stat_data.accesses++;
    } else if (access_type == INST_LOAD) {
        cache_stat_inst.misses++;
        cache_stat_inst.accesses++;
```

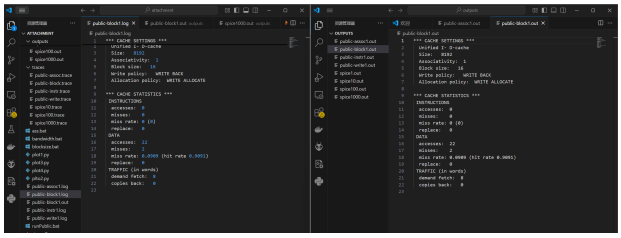
```
    }

if (access_type == DATA_STORE && cache_writealloc==0)
    {
      cache_stat_data.copies_back ++;
      }
else
{    Pcache_line new_line = create_new_cache_line(tag,access_type);

    if (target_cache->set_contents[index] < target_cache->associativity) {
        //未满，将新缓存行插入
        insert(&target_cache->LRU_head[index], &target_cache->LRU_tail[index], new_line);
        target_cache->set_contents[index]++;
        target_cache->contents++;
     if (access_type == DATA_ACCESS || access_type == DATA_STORE) {
          cache_stat_data.demand_fetches += words_per_block;
    } else if (access_type == INST_LOAD) {
     cache_stat_inst.demand_fetches += words_per_block;
    }

    } else {
      //已满，执行替换操作
       Pcache_line lru_tail = target_cache->LRU_tail[index];
        if (cache_writeback && lru_tail->dirty==1) {
            cache_stat_data.copies_back += words_per_block;
        }
         if (access_type == DATA_ACCESS || access_type == DATA_STORE) {
        cache_stat_data.demand_fetches += words_per_block;
        cache_stat_data.replacements ++;}
        else if (access_type == INST_LOAD) {
   cache_stat_inst.demand_fetches += words_per_block;
        cache_stat_inst.replacements ++;
    }
```

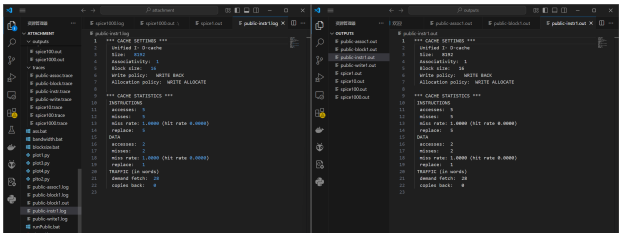
```
        delete(&target_cache->LRU_head[index], &target_cache->LRU_tail[index], lru_tail);
        insert(&target_cache->LRU_head[index], &target_cache->LRU_tail[index], new_line);
    }

    if (access_type == DATA_STORE&&cache_writeback==0) {
          cache_stat_data.copies_back ++; //回写统计
    }
   if (access_type == DATA_STORE && cache_writealloc) {
        new_line->dirty = 1; // 设置新行脏
    }
   }
}
//创建一个新的缓存行
create_new_cache_line(unsigned tag,int access_type) {
    Pcache_line new_line = (cache_line *)malloc(sizeof(cache_line));
    new_line->tag = tag;
    new_line->dirty = (access_type == DATA_STORE);
    new_line->LRU_next = NULL;
    new_line->LRU_prev = NULL;
    return new_line;
}
```

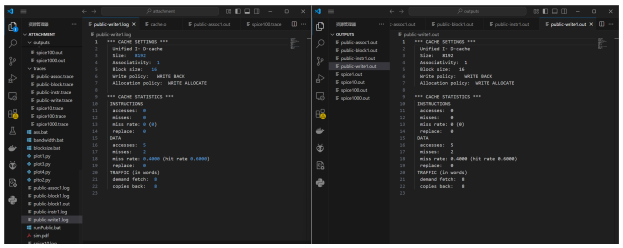
2.4 代码最终测试结果

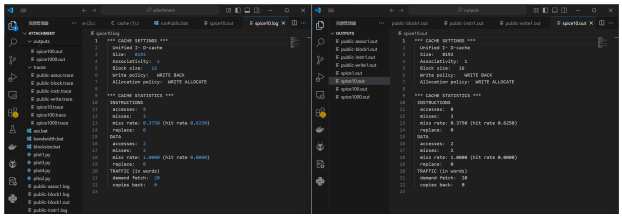
运行 runPublic.bat, 输出结果。

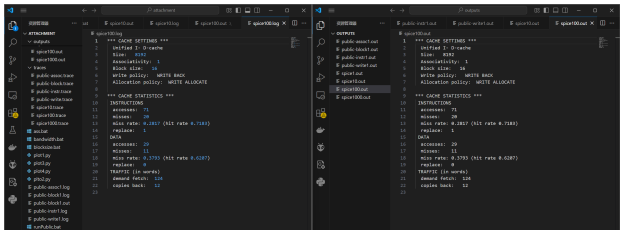


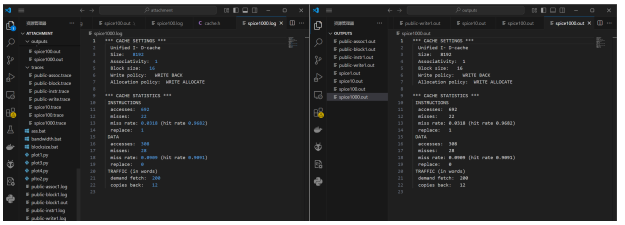












对比输出结果和提供的.out 文件，结果均一致。

3 评估

3.1 Working Set Characterization

在本小节中，我们对 cache performance 和 cache size 的关系进行分析，使用 block 大小为 4 字节的完全关联缓存，使用 write-back cache,write-allocate 策略，初始大小为 4 字节，然后每次增加 2 倍，直到命中率对缓存大小保持不敏感。cache size 的结果可以通过运行脚本文件 workingset.bat 来得到，再使用 plot1.py 绘制下图，其中 x 轴为 cache size 以 2 为底的对数，y 轴为 cache hit rate。

回答问题：

1. 解释一下这个实验在做什么，以及它是如何工作的。此外，请解释命中率与缓存大小图中的特性的重要性。

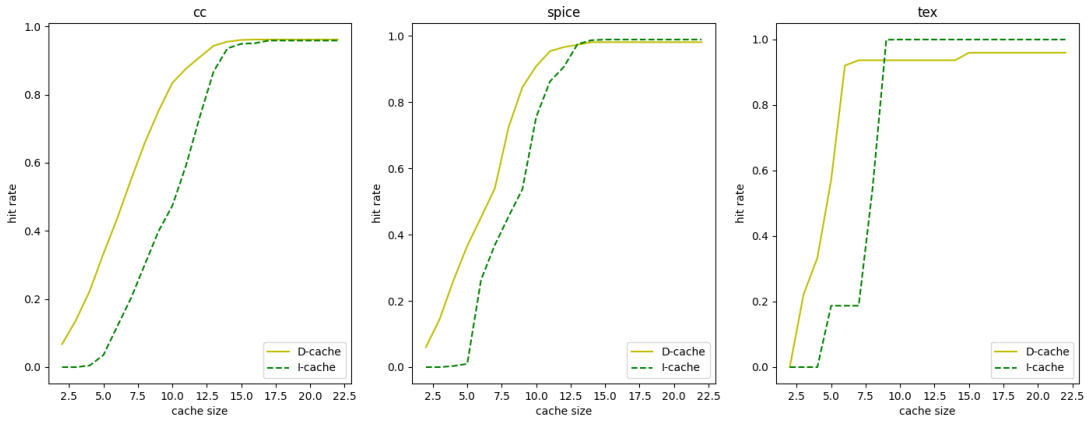


图 1: cache performance 和 cache size 的关系

这个实验在研究 cache 大小对命中率的影响。这个轴是以 2 为底的对数刻度，从 4 到 1,073,741,824 字节，随着缓存大小的增加，命中率增加的越多，因为缓存越大，它包含所需数量的概率就越大; 然而，当 cache size 足够大，达到临界点，无论大小增加多少，命中率已经达到最大值。以及，增长是非线性的，开始时 hit rate 增长快，后面逐渐放缓，至保持定值的情况。

2. 这三个样本跟踪的总指令工作集大小和数据工作集大小是多少？

sample trace	type	working set size
cc	D	242661
cc	I	757341
spice	D	217237
spice	I	782764
tex	D	235168
tex	I	597309

3.2 Impact of Block Size

在本小节中，我们对 cache performance 和 Block size 的关系进行分析，cache 大小为 8 K bytes，使用 write-back cache,write-allocate 策略，关联度为 2，Block size 初始大小为 4 字节，然后每次增加 2 倍，直到增大为 4Kbytes。Block size 的结果可以通过运行脚本文件 blocksize.bat 来得到，再使用 plot2.py 绘制下图，其中 x 轴为 Block size 以 2 为底的对数，y 轴为 cache hit

rate。

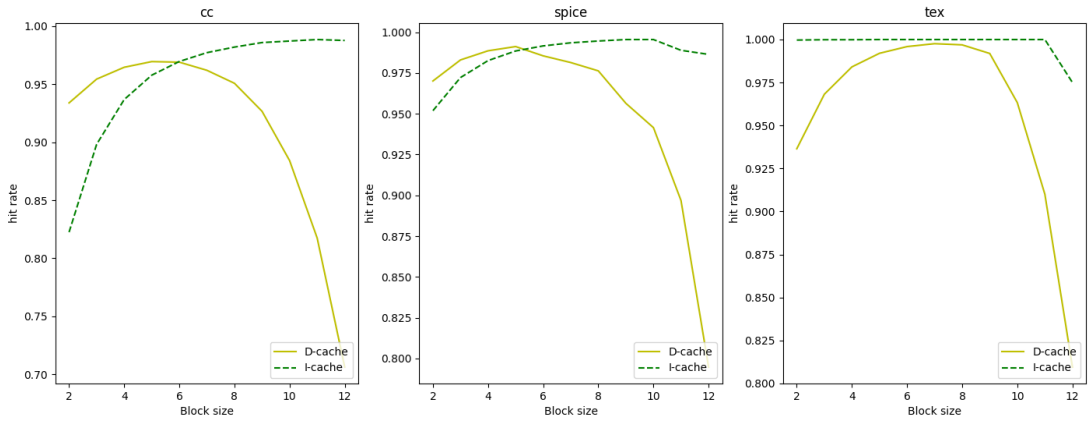


图 2: cache performance 和 Block size 的关系

回答以下问题：

1。解释为什么命中率与块大小图的形状。特别是，解释了空间局部性与这条曲线的形状的相关性。

一开始，block size 越大，hit rate 越高，但是速率在一定点后降低，这些减少在数据缓存的情况下更为明显。其中所有三个文件的 hitrate 都从 block size = 128 左右开始急剧降低。这与空间局部性原理有关。首先，如果块的大小增加，则加载的内存局部数将多于将使用的内存局部数；但如果块大小非常大，具有固定的缓存大小，则更大的块意味着同一缓存中的集较少，因此必须进行更多的替换。

2. 每个跟踪的最佳块大小是多少（分别考虑指令和数据引用）?

对于使用缓存大小的模拟器，每个文件的最佳块大小为：

cc：对于数据缓存大小为 32，对于指令为 2048。

spice：对于大小为 32 的数据缓存，对于指令为 512 或 1024。

tex：对于大小为 128 的数据缓存，对于大小为 32 到 2048 的指令（所有命中率都等于 1）

3. 指令和数据引用的最佳块大小是否不同？这告诉你关于指令引用和数据引用的性质是什么？

数据和指令缓存较优的 block size 是不同的，这告诉我们使用分离的 cache 可以使设计者灵活调整数据和指令缓存的块大小来提高缓存命中率

3.3 Impact of Associativity

在本小节中，我们对 cache performance 和 cache associatity 的关系进行分析，cache 大小为 8 K bytes，使用 write-back cache,write-allocate 策略，Block size 为 128 字节，关联度最初为 2, 然后每次增加 2 倍，直到增大为 64。Associativity 的结果可以通过运行脚本文件 bandwith.bat 来得到，再使用 plot3.py 绘制下图，其中 x 轴为 Associativity 以 2 为底的对数，y 轴为 cache hit rate。

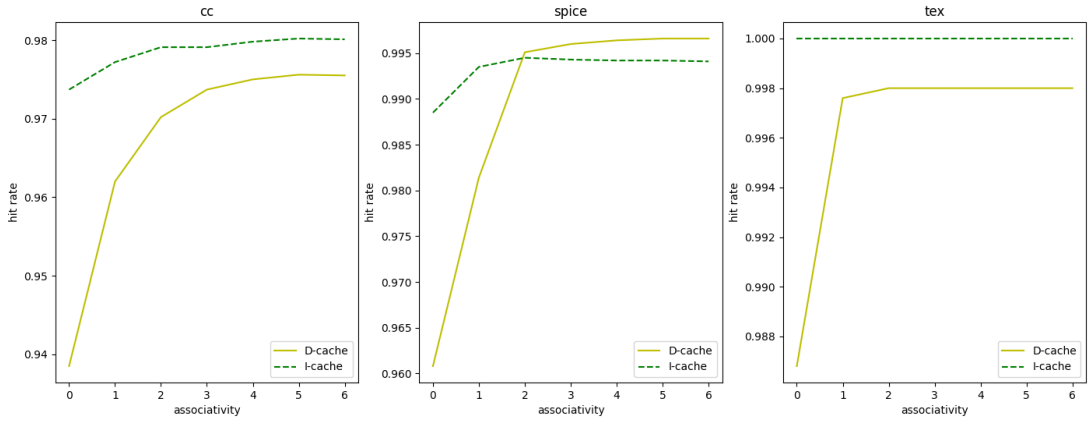


图 3: cache performance 和 cache associatity 的关系

下面进行问题的回答：

1. 解释为什么命中率和联想性图具有这样的形状。

从图中可以看到，associatity 越高，hit rate 越高。因为 associatity 越高，conflict 更少，因此替换和未命中更少。这是有代价的，那就是停止搜索同一行的所有页面以查找与找到要访问的单词的内存块相对应的标签。

2. 指令和数据参考的图之间是否有区别？这告诉了你什么关于联想性对指令和数据引用的影响的区别

数据和指令缓存的命中率曲线是不同的，指令 cache 的 hit rate 总是高于数据 cache。

说明增加关联性对提高数据 cache 的命中率比指令 cache 显著。所以我们在设计 cache 时，要更加注重提高数据 cache 的关联性。

3.4 Memory Bandwidth

回答以下问题：

1。哪个缓存具有较小的内存流量，直写缓存或回写缓存？为什么？

write-back。

可以看出，对于相同的块大小、缓存和关联性，回写策略具有与直写策略完全相同的 demand fetches 数量，但是使用回写策略时，回写的数量会更少。

因为 demand fetch 的操作作为读取的操作，不受写入策略的影响，因此数字相等；在每次写入操作中，直写策略都必须写操作都会同时写入缓存和主内存；而回写策略首先只写入缓存，并将缓存行标记为脏（Dirty）。只有当该缓存行被替换出缓存时，才会将脏数据写回主内存，这意味着在缓存行被替换之前，回写操作只影响缓存，不会产生主内存的写入流量。因此，直写缓存会产生相对较多的内存流量。

2. 在什么情况下，你对上面的问题的回答会翻转？解释原因。

当多核需要共享内存时，直写策略会优于回写策略，因为直写策略能够保持内存中的东西总是最新的，而回写策略则需要较为复杂的机制来实现多核共享内存的问题

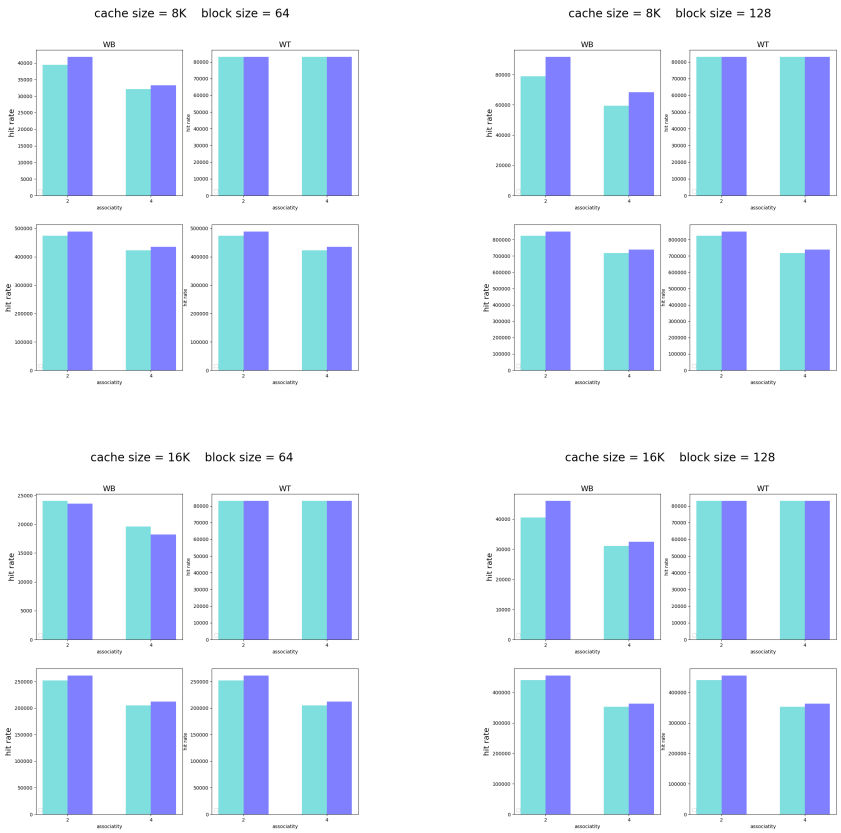


图 4: 文件 cc 的 copies back 和 demand fetches 数量

执行相同的实验，但这次比较写分配和写不分配策略。

通过对比图中数据，可以看出：

当缓存较小时（8K），Write-No-Allocate (WNA) 通常表现更优，特别是在块大小较小（64）或关联性较高（4-way）的情况下，内存流量较低；Write-Allocate (WA) 在数据缺失较少的情况

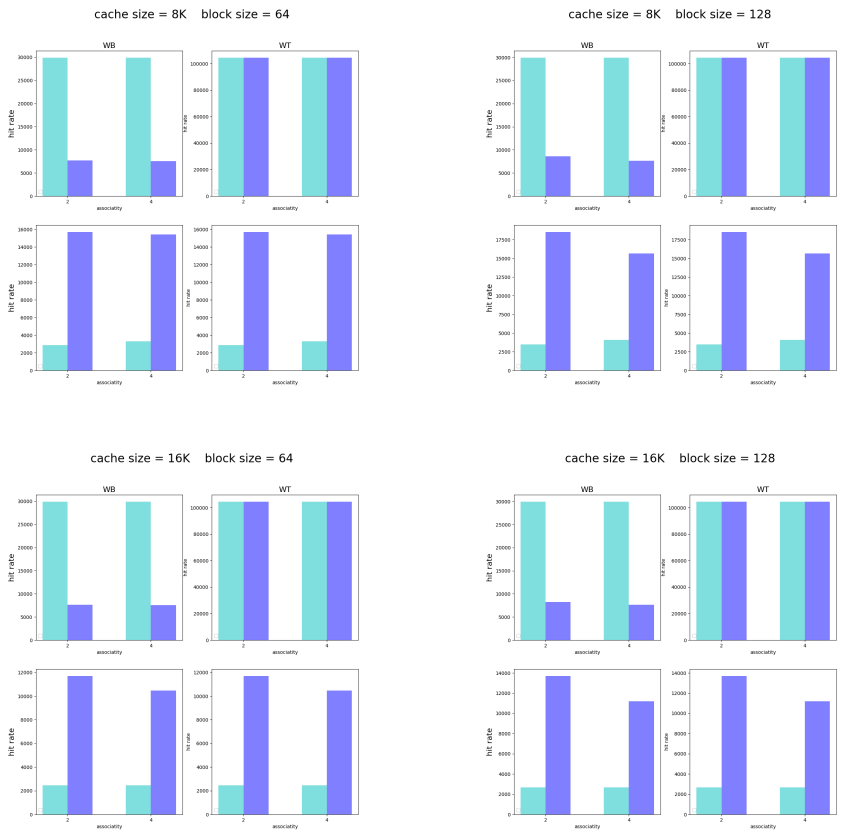


图 5: 文件 tex 的 copies back 和 demand fetches 数量

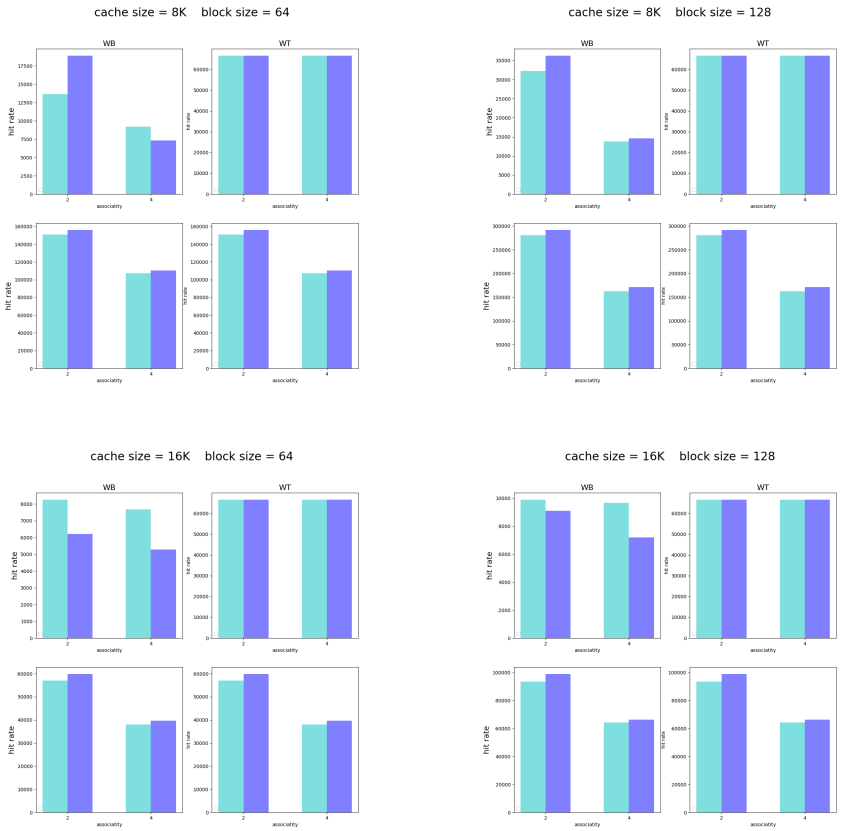


图 6: 文件 spice 的 copies back 和 demand fetches 数量

下可能会更有效，但其 Demand Fetch 明显高于 WNA。当缓存较大时（16K），WNA 的 Demand Fetch 通常更低，但 Copies Back 较高，在大多数情况下，WA 和 WNA 性能相近，具体优劣取决于具体的访问模式。同时，较大的块大小（128）通常导致更高的 Demand Fetch，因为每次 miss 都会带来更大的数据传输。此时 WNA 优势更明显，较小的块大小（64）下，WA 和 WNA 的性能差距可能更小。

回答以下问题：

1。哪个缓存具有较小的内存流量，写分配缓存或无写分配缓存？为什么？

write no allocate。

可以看出，对于相同的块大小，缓存大小和相同的关联性，两者的 copies back 与 cache 大小有关系，没有绝对大小之分，对于 demand fetches，在所有情况下 write-allocate 策略的值都大于 write-not-allocate 策略的值，这是因为在 miss 的情况下，write-allocate 策略会带来一整块内存，然后在其中写入，而 write-no-allocate 只将字直接写入内存。但是 write-allocate 策略具有较少的 data cache miss 和 data cache replace。综合考虑，如果重点关注内存流量，认为 write-allocate 策略更优，尤其是在小缓存（8K）或大块大小（128）的情况下。

2. 在什么情况下，对上面的问题的回答会翻转？解释原因。

如果写操作频繁且重用率高时，write allocate 策略更好。因为如果用 write no allocate 策略对同一块数据进行后续写入或读取操作，这些访问将始终无法命中缓存，因为该块从未被加载到缓存中。这意味着系统需要执行更多的内存访问（将块加载到缓存中）来满足未来的访问需求，从而导致更高的流量。